

Product Requirements Document (PRD)

1. Product Overview

Product Name

RAG CLI

Product Type

Command-line Retrieval-Augmented Generation (RAG) application.

Product Goal

Build a minimal but production-structured RAG application that allows users to ingest local documents and ask questions about their content through a command-line interface.

The primary purpose of this MVP is **learning and understanding the complete RAG pipeline** without hiding core concepts behind high-level frameworks.

The application will use the **Gemini API** for language generation and embeddings, and a local vector database for storing and retrieving document embeddings.

2. Problem Statement

Large Language Models can answer questions based on their general training data, but they do not automatically know the contents of a user's private or local documents.

Users need a way to:

- Add their own documents to an AI system.
- Search those documents semantically.
- Retrieve the most relevant information for a question.
- Generate answers grounded in the retrieved document content.

The goal of this project is to demonstrate how a complete RAG system solves this problem.

3. MVP Objectives

The MVP must provide a clear implementation of the following RAG stages:

1. **Document ingestion**
2. **Document text extraction**
3. **Text chunking**
4. **Embedding generation**
5. **Vector storage**
6. **Semantic similarity search**
7. **Context retrieval**
8. **Answer generation using retrieved context**
9. **Command-line interaction**

The project should prioritize **understanding, clean architecture, and correctness** over feature complexity.

4. Target User

Primary User

Developers or learners who want to understand how a RAG application works internally.

Initial User Context

The user is expected to interact with the application through a terminal and provide documents and questions using CLI commands.

No authentication or multi-user support is required for the MVP.

5. Scope

In Scope

- CLI-based interaction.
- Local document ingestion.
- Support for text-based documents.
- Document chunking.
- Gemini embedding generation.

- Local vector storage.
- Semantic retrieval.
- Gemini-based answer generation.
- Displaying retrieved sources or context references.
- Basic error handling.
- Persistent vector storage.

Out of Scope

The following features are intentionally excluded from the MVP:

- Web UI.
- User authentication.
- Cloud deployment.
- Multi-user support.
- Real-time document synchronization.
- Complex agent workflows.
- Fine-tuning models.
- Hybrid search.
- Reranking models.
- Advanced evaluation pipelines.
- Conversation memory.
- Automatic web search.
- Distributed vector databases.

These can be added after the core RAG system is understood and working correctly.

6. Core User Stories

US-01: Add Documents

As a user, I want to add a document to the RAG system so that its contents can be used to answer future questions.

Acceptance Criteria

- The user can provide a local document path through the CLI.
- The application validates that the file exists.
- The document text is extracted successfully.
- The text is split into chunks.

- Embeddings are generated for each chunk.
 - The chunks and embeddings are stored in the vector database.
 - The CLI displays a successful ingestion message.
-

US-02: Ask Questions

As a user, I want to ask a question about my documents so that I can receive an answer based on their content.

Acceptance Criteria

- The user can submit a natural-language question.
 - The application generates an embedding for the question.
 - Relevant document chunks are retrieved.
 - Retrieved context is sent to Gemini with the user's question.
 - The system returns a generated answer.
 - The answer is displayed clearly in the terminal.
-

US-03: View Sources

As a user, I want to know which document content was used to generate an answer so that I can verify the information.

Acceptance Criteria

- The system stores metadata about the original document.
 - Retrieved chunks contain source metadata.
 - The CLI displays relevant source information with the answer.
 - At minimum, the original filename and chunk reference should be available.
-

7. Functional Requirements

FR-01: Document Ingestion

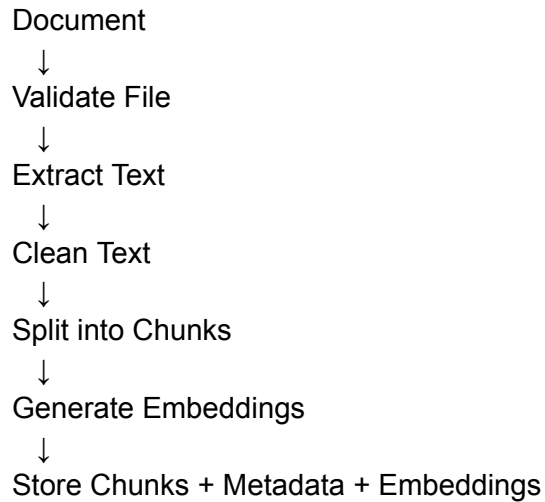
The application must support ingesting documents from the local file system.

Initial support should focus on one or more simple text-based formats, such as:

- `.txt`
- `.md`

PDF support may be added later as an extension.

Process



FR-02: Text Chunking

Documents must be divided into smaller chunks before embedding.

Each chunk should contain:

- The chunk text.
- A unique chunk ID.
- The source document name.
- Chunk position or index.

The chunking implementation should support configurable:

- Chunk size.
- Chunk overlap.

Initial Strategy

Use a simple character-based or word-based chunking strategy for the MVP.

Example:

Chunk 1: Characters 0–1000
Chunk 2: Characters 800–1800
Chunk 3: Characters 1600–2600

The overlap helps preserve context between adjacent chunks.

FR-03: Embedding Generation

The application must generate vector embeddings for every document chunk using a Gemini embedding model.

The same embedding model should be used for:

- Document chunks.
- User queries.

Each embedding represents the semantic meaning of its text and is used for similarity search.

FR-04: Vector Storage

The application must persist document chunks and embeddings in a local vector database.

Each stored record should include:

ID
Embedding
Document Text
Source Metadata
Chunk Index

The vector database must remain available between application runs.

The MVP should use **ChromaDB** as the initial local vector database.

FR-05: Semantic Retrieval

When the user asks a question:

1. Generate an embedding for the question.
2. Search the vector database.
3. Retrieve the most semantically relevant chunks.
4. Return the top K results.

The value of K should be configurable.

Initial default:

Top K = 3

FR-06: Answer Generation

The retrieved chunks must be combined into a context prompt and sent to Gemini.

The model must be instructed to:

- Answer based primarily on the provided context.
- Avoid inventing information that is not present in the context.
- Clearly state when the answer cannot be found in the available context.

Example Prompt Structure

You are a helpful assistant answering questions based on the provided context.

Context:

{retrieved_chunks}

Question:

{user_question}

Instructions:

- Answer using the provided context.
- If the answer is not available in the context, say that you do not have enough information.
- Do not make up information.

Answer:

FR-07: CLI Interface

The application must provide simple CLI commands.

Initial Command Set

Ingest a Document

```
python main.py ingest ./documents/example.txt
```

Ask a Question

```
python main.py ask "What is this document about?"
```

Interactive Mode

An optional interactive mode may allow:

```
RAG CLI > What are the main topics in the document?
```

Answer:

...

Sources:

- example.txt

Interactive mode can be implemented after the command-based MVP is functional.

8. Non-Functional Requirements

NFR-01: Simplicity

The implementation must prioritize understandable code over unnecessary abstractions.

The first version should avoid frameworks that completely hide the RAG pipeline.

NFR-02: Modularity

The codebase must separate responsibilities, including:

- Configuration.
- Document loading.
- Chunking.
- Embedding generation.
- Vector storage.

- Retrieval.
 - Generation.
 - CLI interaction.
-

NFR-03: Maintainability

The project should follow standard Python practices:

- Clear naming.
 - Small focused modules.
 - Type hints where appropriate.
 - Environment variables for secrets.
 - No API keys hardcoded in source code.
-

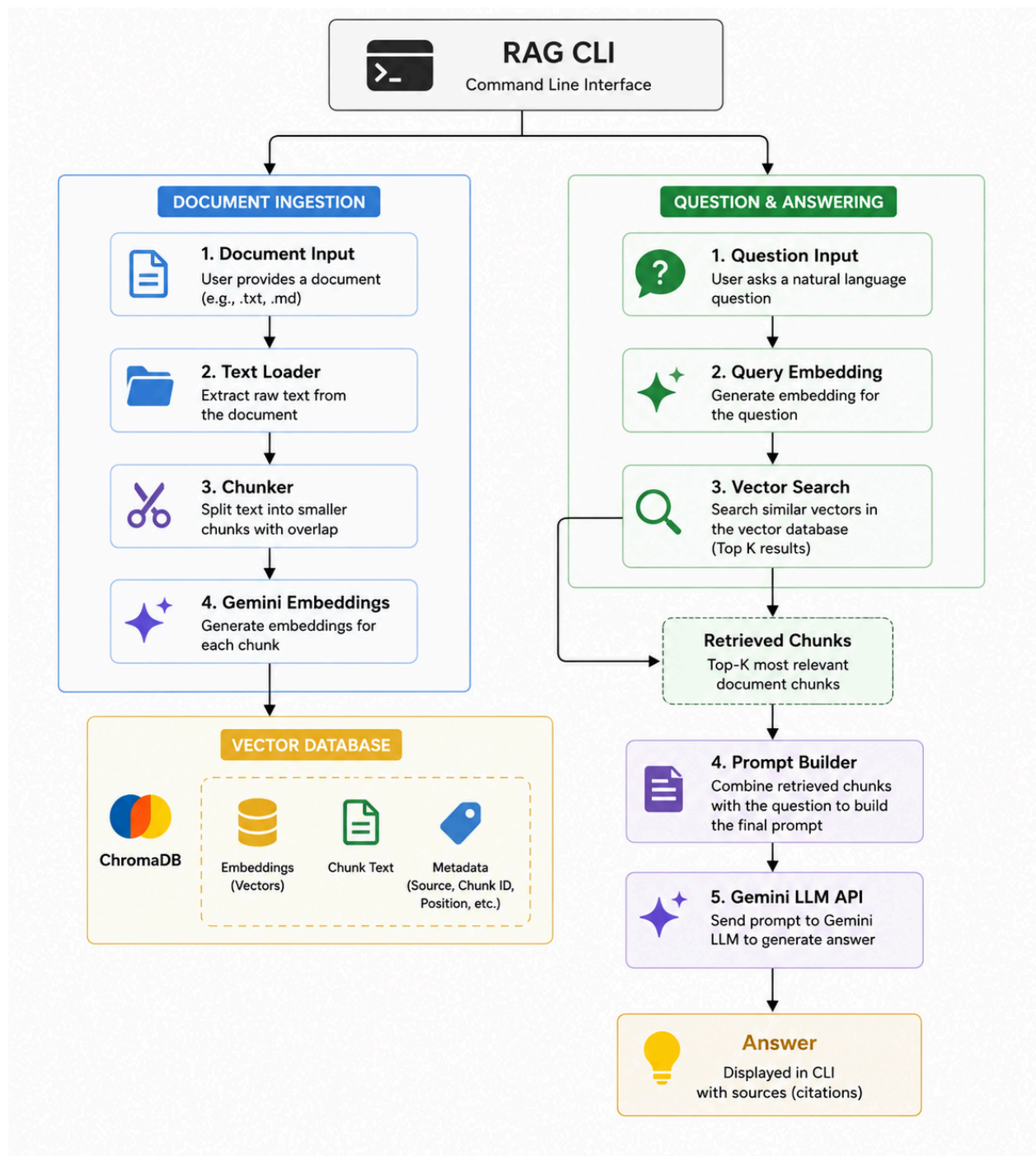
NFR-04: Error Handling

The application must handle common errors gracefully:

- File not found.
 - Unsupported file format.
 - Empty document.
 - Missing API key.
 - API request failure.
 - Vector database errors.
 - No relevant documents found.
-

9. System Architecture

High-Level Architecture



10. Technology Stack

Layer	Technology
Programming Language	Python
LLM	Gemini API
Embeddings	Gemini Embedding Model
Vector Database	ChromaDB
Configuration	Environment Variables
CLI	Python standard library or Typer
Dependency Management	uv or pip
Testing	Pytest

Principle

The project should use as few dependencies as necessary to keep the RAG pipeline visible and understandable.

High-level RAG orchestration frameworks should **not** be used in the first version.

11. Proposed Project Structure

```
None
simple-rag/
|
├─ app/
|   ├── __init__.py
|   ├── config.py
|   |
|   └─ loaders/
```


None

graph TD

```
A[User selects document] --> B[Validate document]
B --> C[Extract text]
C --> D[Split into chunks]
D --> E[Generate embedding for each chunk]
E --> F[Store chunks and metadata in ChromaDB]
```

B. Question Answering Flow

None

graph TD

```
A[User enters question] --> B[Generate query embedding]
B --> C[Search similar document embeddings]
C --> D[Retrieve top relevant chunks]
D --> E[Build context]
E --> F[Send context + question to Gemini]
F --> G[Generate answer]
G --> H[Display answer and sources]
```

13. Prompt Requirements

The answer-generation prompt must contain:

- A system-level instruction.
- Retrieved document context.
- The user's question.
- Clear grounding instructions.

The system should not present retrieved information as guaranteed truth. It should answer based on the available context.

If retrieval produces insufficient information, the model should communicate that limitation rather than hallucinating.

14. CLI Output Requirements

A successful question should produce output similar to:

Question:

What are the requirements for this project?

Answer:

The project requires ...

Sources:

1. project-requirements.txt — Chunk 3
2. project-requirements.txt — Chunk 4

Errors should be understandable:

Error: Document not found.

Please check the file path and try again.

15. Success Criteria

The MVP will be considered successful when a user can:

1. Start the application locally.
 2. Add a document successfully.
 3. Persist the document embeddings.
 4. Restart the application without losing indexed documents.
 5. Ask a question related to the document.
 6. Retrieve relevant chunks from the vector database.
 7. Generate an answer grounded in those chunks.
 8. See the sources used for the answer.
 9. Receive understandable errors for common failures.
-

16. MVP Milestones

Milestone 1 — Project Setup

- Create Python project.
- Configure environment variables.
- Configure Gemini API client.
- Create basic CLI entry point.

Milestone 2 — Document Processing

- Implement document loading.
- Implement text cleaning.
- Implement chunking.

Milestone 3 — Embeddings and Storage

- Generate embeddings with Gemini.
- Configure ChromaDB.
- Store chunks and metadata.

Milestone 4 — Retrieval

- Generate query embeddings.
- Perform semantic similarity search.
- Retrieve top relevant chunks.

Milestone 5 — Generation

- Build the RAG prompt.
- Send context and question to Gemini.
- Generate grounded answers.

Milestone 6 — CLI Experience

- Add `ingest` command.
- Add `ask` command.
- Display sources.
- Improve error handling.

Milestone 7 — Testing and Documentation

- Add basic unit tests.
 - Document setup instructions.
 - Document the RAG architecture.
 - Add usage examples.
-

17. Future Enhancements

After completing the MVP, potential improvements include:

- PDF and DOCX support.
 - Recursive or semantic chunking.
 - Metadata filtering.
 - Hybrid keyword + vector search.
 - Reranking retrieved results.
 - Conversation history.
 - Streaming responses.
 - Web-based interface.
 - REST API with FastAPI.
 - Document management commands.
 - Evaluation datasets and automated RAG quality testing.
 - Docker deployment.
 - Multi-user support.
-

18. Key Learning Outcomes

By completing this project, the developer should understand:

- Why RAG is needed.
 - How embeddings represent semantic meaning.
 - Why documents must be chunked.
 - How vector similarity search works.
 - What a vector database stores.
 - The difference between retrieval and generation.
 - How context is injected into an LLM prompt.
 - Why hallucinations can still occur in RAG systems.
 - How RAG architecture can be improved beyond a basic MVP.
-

19. Final MVP Definition

The **Simple RAG CLI MVP** is complete when a user can ingest local documents, store their semantic embeddings locally, ask natural-language questions, retrieve relevant document chunks, and receive Gemini-generated answers grounded in that retrieved context.

The project intentionally focuses on the **fundamental RAG pipeline** rather than building a feature-heavy application. Every major stage of the system should remain visible in the codebase and understandable independently.